

Reto 2 — Patrones de Diseño Arquitectónico

Del diseño correcto (SOLID) al diseño robusto (SOLID + Patrones) · Sistema Hacienda

Equipo: Abel Garcia · Miguel Moreno · Juan Esteban Vasco · Juan Jose Mesa

Solicitud de cambio implementada: SC-1 — venta de productos derivados del ganado (lácteos, carne, piel)

Línea base (AS-IS): entrega del Reto 1, Reto_Hacienda_Modernizacion_SOLID/03-src/Modificado/

Código del TO-BE: src/Modificado/

Índice

#	Sección	Actividad	Entregable de la rúbrica
1	Contexto, alcance y límites	—	—
2	Análisis de puntos de dolor	1	Tabla de seis puntos con costo medido
3	Tabla de decisión de patrones	2.1	Diez patrones evaluados, cuatro adoptados
4	Bitácora de decisiones frente a la IA	2.2	Quince decisiones registradas
5	Diseño TO-BE · lo que sale y lo que entra	3.1	Diagrama de dos capas
6	Tabla de cambio estructural	3.2	Treinta y dos elementos
7	Fichas de los patrones adoptados	3.3	Una ficha por patrón
8	Matriz de verificación SOLID	4.1	Cuatro patrones × cinco principios
9	Evidencia de comportamiento preservado	4.2	Salidas antes y después
10	Registro de riesgos y plan de cambio	5	Seis riesgos con señal de alerta
11	Vista para el negocio	6	—
12	Vista para el equipo de desarrollo	6	Guía de dónde tocar

1. Contexto, alcance y límites

El Reto 1 dejó un sistema **correcto**: las responsabilidades quedaron separadas y las dependencias invertidas. Lo que no resolvió es **quién decide qué implementación concreta se usa y dónde se ensambla el sistema**. Ese es el trabajo de este trimestre.

El punto de partida ya traía patrones aplicados sin nombrarlos —IFabricaRes con tres implementaciones es un Factory Method funcionando—, así que la tarea no fue introducir un vocabulario nuevo sino medir dónde el diseño seguía siendo caro y extender lo que ya funcionaba.

Se adoptaron **cuatro patrones** —Factory Method, Strategy, Observer y Composite—, cada uno anclado a un punto de dolor medido, y se implementó **SC-1**. Los tres límites del encargo se respetan: el comportamiento observable queda congelado y verificado con diff vacío (§9); no se cambia el estilo arquitectónico ni se introduce framework alguno; y ningún principio SOLID queda roto (§8).

2. Actividad 1 · Análisis de puntos de dolor

Un punto de dolor aquí no es una violación de SOLID —esas se cerraron el trimestre pasado—, sino un sitio donde el diseño es correcto y aun así cambiarlo obliga a abrir muchos archivos, o donde el compilador no avisa si alguien se olvida de uno.

Cómo medimos el costo. Para cada punto elegimos un cambio concreto, buscamos en todo el código el tipo o la cadena afectada y seguimos las llamadas hasta sus extremos. La cifra es el número de clases y archivos que hay que abrir; las vistas .cshtml se cuentan aparte porque el compilador no las verifica.

Criterio de prioridad. *Alta*: bloquea una solicitud del Anexo B o cuesta 10 archivos o más. *Media*: entre 5 y 9 archivos, o tiene puntos de cambio que el compilador no protege. *Baja*: menos de 5.

ID	Dónde	Qué lo hace rígido o caro	Escenario de cambio y costo hoy	Prioridad	Origen
P-01	FabricaVacunas.cs, VacunaService.cs:49-58, VacunaController.cs:95-107, MapeadorVacuna.cs, ServicioVacunacion.cs:60-84, Res.cs:168-171	Cada tipo concreto de vacuna está escrito a mano en toda la cadena: un método por tipo en la fábrica, un if/else en el servicio, otro en el controlador, cuatro ramas en el mapeador y dos contadores en la vacunación. Hasta Res declara MaxVacunasBacterianas y MaxVacunasVivas	Un tercer tipo de vacuna: 10 clases y 4 vistas = 14 archivos , más 1 clase nueva	Alta	Conteo verificado con IA
P-02	Venta.cs:13,16, ServicioVenta.cs:42-46, ValidarVenta.cs:14, MapeadorVenta.cs:26	Una Venta exige una Res: campo obligatorio del constructor, el validador la rechaza si es nula y la línea de disco escribe sus cuatro datos. Además vender borra el animal del potrero	SC-1, vender derivados: 12 clases y 2 vistas = 14 archivos . Hoy la solicitud es inviable sin tocarlas todas	Alta	Hallazgo propio

ID	Dónde	Qué lo hace rígido o caro	Escenario de cambio y costo hoy	Prioridad	Origen
P-03	Potrero.cs:21-24, 105-108, GestorReses.cs:30-31, 77-78, 188-189, ServicioVacunacion.cs:29-30	Los publicadores de avisos se crean con new dentro del dominio: 8 instanciaciones en 3 clases. No hay interfaz común y la secuencia de disparo está escrita a mano	Cambiar la política de avisos: 3 clases de dominio y 5 métodos , más 8 instanciaciones que ninguna prueba puede sustituir	Media	Hallazgo propio
P-04	Potrero.cs:15, ResService.cs:233-238, MapeadorRes.cs:155-156, 3 vistas	Un cuarto tipo de res obliga a añadir un valor al enum que vive dentro de la entidad Potrero, más un diccionario de presentación y tres vistas	Un cuarto tipo de res: 3 clases y 3 vistas = 6 archivos , más 2 clases nuevas. 4 de los 6 no los verifica el compilador	Media	Hallazgo propio
P-05	GuardadoValidado.cs:49-65, 69-149, ValidarRes.cs:19, ResultadoValidacion.cs:23	GuardadoValidado recibe 7 dependencias y repite 5 veces el bucle validar → cortar → persistir. Cada validador mete todas sus reglas en un solo if que devuelve siempre el mismo texto	Un agregado nuevo (producto de SC-1, historia clínica de SC-3): 2 clases nuevas, 2 archivos modificados y 3 servicios que recompilan por el cambio de firma del constructor	Media	Conteo verificado con IA
P-06	MapeadorRes.cs, MapeadorVacuna.cs, MapeadorVenta.cs, MapeadorPotrero.cs, RepositorioUsuariosArchivo.cs	La persistencia lee por posición de columna: 33 accesos partes[n] detrás de guardas que descartan la línea sin lanzar excepción	Añadir un campo a cualquier entidad persistida: 4 mapeadores y 4 repositorios = 8 archivos , con riesgo de pérdida silenciosa del histórico	No se interviene	Conteo verificado con IA

P-01. Tres capas resuelven lo mismo de tres maneras —VacunaController.cs:95 compara una cadena, VacunaService.cs:49 mira qué parámetros opcionales vienen llenos y MapeadorVacuna.cs:73, 140 lee la quinta columna—, y lo más caro está en Res.cs:168-171: dos miembros abstractos con el nombre de una subclase concreta, así que un tercer tipo de vacuna termina cambiando la jerarquía del ganado.

P-02. Vender leche o piel con este flujo daría de baja la vaca:

```
// ServicioVenta.cs:42-46 (AS-IS)
Venta venta = new Venta(potrero, DateTime.Now, res, monto);
_hacienda.L_ventas.Add(venta);
_hacienda.L_potreros.Where(p => p == potrero).FirstOrDefault().L_reses.Remove(res);
```

El ADR §3.3.3 del Reto 1 diseñó una venta polimórfica y proyectó 7 archivos para SC-1, pero nunca se implementó porque el trimestre pasado se ejecutó SC-2: el recuento real sobre el código es 14.

P-03. Seis firmas distintas —Informar_Peso_Min(Res, IReceptorEventos) frente a Informar_Potrero_Mitad(ushort, Potrero, IReceptorEventos)— impiden que exista una colección sobre la que iterar, así que la secuencia se escribe a mano en cinco métodos y su orden es comportamiento observable (Potrero.cs:102-108).

P-06 · por qué no se interviene. La cura real —cabecera, versión o un serializador— reescribe los seis archivos de datos del cliente, que es salida observable y está prohibido; y aun conservando el formato byte a byte habría que tocar los 8 archivos de Infraestructura para producir lo mismo que hoy. La regla vigente —columnas nuevas solo al final, un registro de forma distinta va a un archivo nuevo (MapeadorRes.cs:37-40)— contiene el problema con costo cero y ya sobrevivió a SC-2 sin perder una línea. El problema cuesta hoy cero archivos; el remedio cuesta ocho y arriesga los datos del cliente.

Patrón común. Cinco de los seis puntos responden a lo mismo: el Reto 1 resolvió quién hace qué, pero no quién decide qué objeto concreto se usa.

3. Actividad 2.1 · Tabla de decisión de patrones

Diez patrones evaluados, cuatro adoptados. La solicitud implementada es **SC-1** porque aterriza sobre P-02, el punto de mayor costo medido.

Patrón	Familia	Dolor	Qué gana y qué cuesta	Decisión	Por qué
Factory Method	Creador	P-01	Gana: un registro IFabricaVacuna con una implementación por tipo borra los cuatro métodos fijos y los tres puntos de decisión; un tipo nuevo pasa de 10 archivos a 1 clase y 1 línea. Cuesta: dos clases más, un salto de indirección, y no cierra los 4 archivos de conteo (Res.cs:168-171 y sus subtipos)	Adoptado	El sistema ya tiene el patrón funcionando en IFabricaRes con tres implementaciones: aplicamos a las vacunas la forma probada en el ganado, no una nueva

Patrón	Famili-a	Dolor	Qué gana y qué cuesta	Deci-sión	Por qué
Strategy	Com-porta-miento	P-02	Gana: vender un animal aplica el retiro del potrero y vender leche, carne o piel no toca el inventario, sin duplicar el método. Cuesta: una interfaz, dos implementaciones, y el efecto de una venta se lee en la raíz y no en el método	Adop-tado	El Remove(res) de ServicioVenta.cs:46 es una política de inventario escrita dentro de la operación de venta; separarla es lo único que hace viable vender algo distinto del animal
Observer	Com-porta-miento	P-03	Gana: una interfaz común sobre un contexto único borra las 8 instanciaciones del dominio y deja un aviso nuevo en 1 clase y 1 línea. Cuesta: un contexto que algunos avisos ignoran, y el orden de disparo pasa a depender del orden de registro	Adop-tado	Cubre 5 de los 6 publicadores. PublisherVacunaVencida queda fuera: devuelve bool y ServicioVacacion.cs:95 lanza según ese valor, así que es guarda y no notificación
Composite	Es-tractu-ral	P-05	Gana: ValidadorCompuesto<T> agrupa IValidador<T> y corta en el primero que falla; una regla nueva es una clase. Cuesta: más clases pequeñas y depurar exige mirar cómo se compuso en la raíz	Adop-tado	Ataca la mitad de P-05 que necesita patrón; la otra mitad —7 dependencias y 5 métodos iguales— se resuelve con un método genérico, que no es un patrón
Abstract Factory	Crea-cional	P-01	Una interfaz de familia y una fábrica por familia, con una sola familia detrás	Des-carta-do	Solo existe el catálogo de vacunas: la interfaz quedaría con una única implementación, que es indirección sin variación. Se justificaría con esquemas sanitarios distintos por especie
Builder	Crea-cional	P-01	Resolvería los parámetros opcionales de las cuatro sobrecargas de creación	Des-carta-do	Las sobrecargas cruzan dos ejes independientes —tipo × unidad o lote— y Builder solo ataca uno. Además reescribiría los ArgumentException que distinguen nameof(lote) de nameof(lote_base), texto que llega al operario
Chain of Responsibility	Com-porta-miento	P-05	Encadenaría las reglas pasando el objeto de una a la siguiente	Des-carta-do	Significa «el primero que pueda resolver»; aquí todas las reglas aplican al mismo objeto y el corte es una optimización, no un significado. Volvería significativo un orden que hoy no lo es
Facade	Es-tractu-ral	P-02, P-05	Un punto de entrada único a los servicios de aplicación	Des-carta-do	Ese punto ya existe: las clases *Service y GuardadoValidado. Una capa encima es cambio de estilo arquitectónico, fuera del encargo, y tiende a absorber lógica hasta romper SRP
Decorator	Es-tractu-ral	P-05	Envolvería cada validador para añadir reglas sin modificarlo	Des-carta-do	Paga más que Composite: el orden de anidamiento pasa a importar y hay que leer la pila entera. Su riesgo es cambiar el contrato envuelto, y aquí ese contrato es ResultadoValidacion, cuyo texto está congelado
Proxy	Es-tractu-ral	P-05	Interceptaría las llamadas para validar antes de persistir	Des-carta-do	Es lo que el sistema original hacía con Castle DynamicProxy y ADR-03 desmontó, porque nadie podía sustituir la validación desde fuera. Aspectos/InterceptorAutenticacion.cs quedó sin llamadores. Reintroducirlo es volver a un framework que resuelve el problema por nosotros

Builder frente a Factory Method (P-01). Fue la decisión más cerrada: cuatro métodos con firmas largas y validación repetida es el síntoma clásico de Builder. Lo que lo descarta es mirar qué varía: no varía el número de parámetros, varían dos cosas independientes —qué tipo y si es unidad o lote—. Un Builder resolvería la primera con un `if` interno y dejaría la segunda igual, y su API fluida unificaría los dos parámetros de lote, cambiando un mensaje de error que es salida observable.

Chain of Responsibility frente a Composite (P-05). Los dos producen una lista de validadores y permiten agregar reglas sin tocar código, así que la diferencia no está en el resultado sino en lo que el patrón declara. La cadena declara «alguien se hará cargo»; el compuesto, «todos estos son, juntos, la validación de este tipo». Lo segundo es lo que ocurre: nulidad, nombre, peso y edad son la definición de res válida.

Puntos sin patrón. P-04 no lo recibe porque el enum `l_tipos_potreros` se persiste literalmente —Potreros.txt contiene `Potrero_Terneros|ternero` y `MapeadorPotrero.cs:31` lo reconstruye con `Enum.Parse`—, así que eliminarlo cambiaría el archivo de datos del cliente. **P-06** quedó descartado en §2 con el mismo argumento.

Por eso no hay un quinto patrón: tendría que atacar uno de los dos y sería un patrón sin punto rígido que lo justifique.

4. Actividad 2.2 · Bitácora de decisiones frente a la IA

ID	Qué consultamos	Qué propuso la herramienta	Qué hicimos	Argumento propio y evidencia
B-01	Qué solicitud implementar, SC-1 o SC-3	SC-3, por ser más acotada	Corregimos	SC-1 aterriza sobre P-02, el punto de mayor costo medido (14 archivos); SC-3 cae sobre P-05, de prioridad Media. Una solicitud acotada dejaría el punto caro sin tocar
B-02	Cuántos archivos cuesta la persistencia posicional de P-06	Contó 9: 5 mapeadores y 4 repositorios	Corregimos	RepositorioUsuariosArchivo estaba contado dos veces. Infraestructura/ tiene 8: 4 mapeadores y 4 repositorios. Es la cifra que sostiene el argumento de no intervenir
B-03	Qué patrón resuelve la creación de vacunas de P-01	Abstract Factory	Rechazamos	Solo hay una familia; la interfaz quedaría con una implementación. Factory Method replica lo que IFabricaRes ya hace con tres implementaciones reales
B-04	Cómo unificar las cuatro sobrecargas de FabricaVacunas	Builder con API fluida	Rechazamos	Cruzan dos ejes independientes y Builder ataca uno. FabricaVacunas.cs:53 y :118 lanzan con nameof(lote) y nameof(lote_base), y ese nombre viaja en el mensaje que ve el operario
B-05	Qué patrón usar para las reglas de validación de P-05	Chain of Responsibility	Corregimos	La cadena significa «el primero que pueda resolver»; las cuatro condiciones de ValidarRes.cs:19 aplican todas. Adoptamos Composite y conservamos el corte como optimización
B-06	Si conviene una fachada sobre los servicios	Facade, para simplificar la entrada	Rechazamos	Los *Service ya son ese punto de entrada. Una capa más es estilo arquitectónico, y GuardadoValidado ya muestra con sus 7 dependencias la tendencia a absorber lógica
B-07	Cómo aplicar Observer a los seis publicadores	Una interfaz común para los seis	Corregimos	Cubrimos cinco. PublisherVacunaVencida.cs:11 devuelve bool y ServicioVacunacion.cs:95 lanza según ese valor: meterlo en el patrón cambiaría cuándo se lanza la excepción
B-08	Si GuardadoValidado necesita patrón para sus 5 métodos iguales	Command, un objeto por operación	Rechazamos	Los cinco difieren solo en el tipo: un método genérico los unifica. Command sumaría cinco clases para expresar algo que el lenguaje ya expresa, que es la sobre-ingeniería que el enunciado penaliza
B-09	Si el enum de P-04 se puede eliminar con un registro de tipos	Sí, un registro de cadenas	Rechazamos	El enum se persiste literalmente y MapeadorPotrero.cs:31 lo reconstruye con Enum.Parse: cambiarlo reescribe el archivo de datos del cliente
B-10	Cómo modelar la venta de derivados para SC-1	Jerarquía de Venta por tipo de producto	Corregimos	Venta sigue siendo un solo tipo con el artículo dentro. Una jerarquía obligaría a un discriminador en Ventas.txt, y MapeadorVenta.cs:26 escribe siete columnas por posición
B-11	Si el conteo de 14 archivos de P-02 era correcto	Tomó como válida la cifra del ADR (7)	Corregimos	La Venta polimórfica del ADR §3.3.3 nunca se implementó porque se ejecutó SC-2. Venta.cs:16 sigue exigiendo una Res; recontamos sobre el código
B-12	Cuántos patrones adoptar	Entre tres y cinco, sin recomendación	Fue idea nuestra	Cuatro, uno por cada punto priorizado que admite patrón: P-01, P-02, P-03 y P-05. Un quinto tendría que atacar P-04 o P-06, descartados con argumento de costo
B-13	Cómo debía quedar la estrategia de venta de P-02	IEfectoVenta con retiro de inventario y sin efecto	Aceptamos	Verificamos antes que ServicioVenta.cs:46 es el único punto que modifica el inventario al vender; si hubiera habido un segundo, la interfaz habría nacido incompleta
B-14	Cómo conservar el orden de disparo de los avisos	Registrarlos en orden en la raíz e iterar	Aceptamos	El orden mitad → lleno → peso mínimo → peso de venta de Potrero.cs:105-108 es salida observable; iterar la colección lo reproduce sin código de ordenamiento
B-15	Si convenía Decorator en vez de Composite	Decorator, por ser el habitual para añadir comportamiento	Rechazamos	Envolver hace significativo un orden de anidamiento que aquí no lo es, y el contrato envuelto es ResultadoValidacion, congelado. Una envoltura que lo cambie rompe LSP para el llamador

5. Actividad 3.1 · Diseño TO-BE · lo que sale y lo que entra

Diagrama único de dos capas superpuestas, hecho en draw.io. Archivo fuente: 03-diseno-tobe/Diagrama_AsIs_ToBe_Final.drawio.

Leyenda. Rojo: elementos del AS-IS que salen o cambian de responsabilidad. Verde: elementos que entran, con el patrón al que pertenecen y el papel que cumplen dentro de él. Negro: lo que se conserva sin cambios.

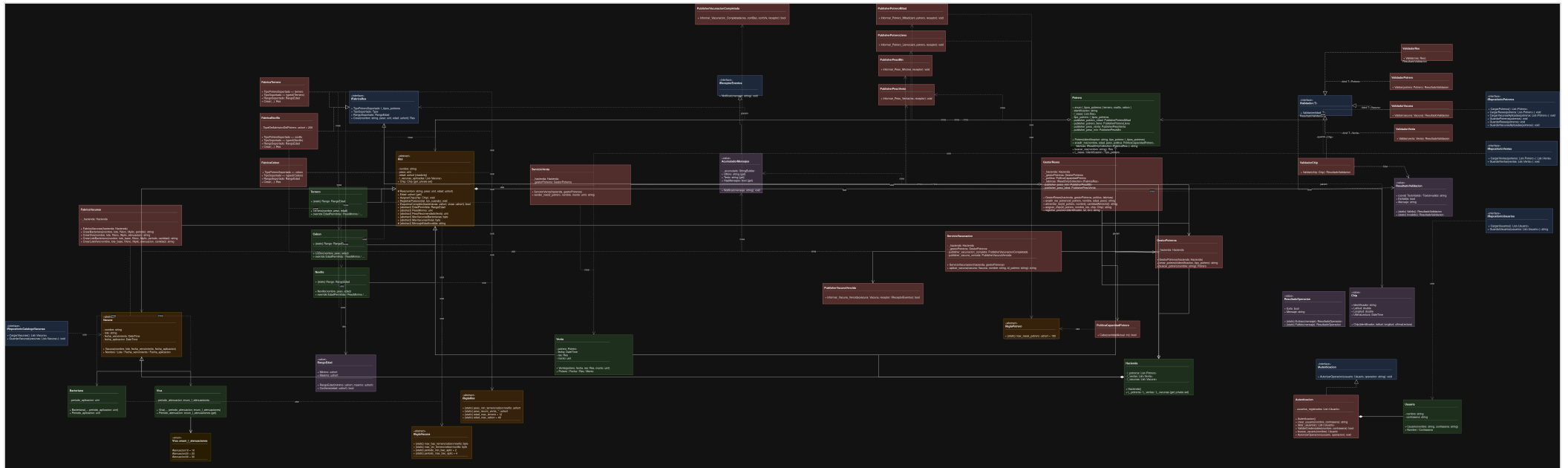


Figura 1. Recorte del diseño del Reto 1 con los elementos marcados: `ValidadorRes` (única clase que desaparece), los cuatro métodos fijos de `FabricaVacunas`, la línea `Remove(res)` dentro de `vender_res`, las 8 instanciaciones con `new` de `publicadores` en `Potrero`, `GestorReses` y `ServicioVacunacion`, y las tres ramificaciones por tipo de vacuna en `servicio`, `controlador` y `mapeador`.

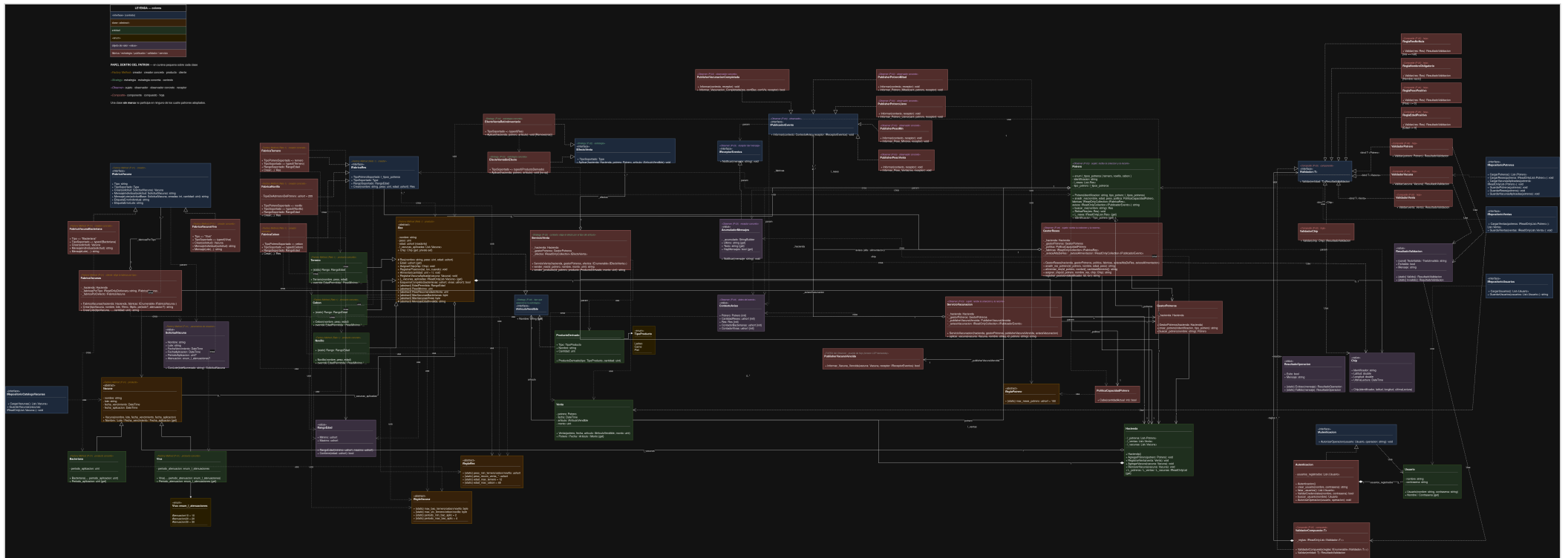


Figura 2. TO-BE con cada clase rotulada por patrón y papel: IFabricaVacuna (Factory Method · creador) con FabricaVacunaBacteriana y FabricaVacunaViva (creadores concretos); IEfectoVenta (Strategy · estrategia) con EfectoVentaRetiroInventario y EfectoVentaSinEfecto; IPublicadorEvento (Observer · observador) con los cinco publicadores; ValidadorCompuesto<T> (Composite · compuesto) con las cuatro reglas de ReglasRes/ como hojas. RaizComposicion aparece como el único punto donde se elige una implementación para los cuatro patrones.

6. Actividad 3.2 · Tabla de cambio estructural

Inventario del TO-BE frente a la línea base, obtenido comparando los dos árboles de código: **1 elemento sale, 17 entran y 23 se transforman**. Las filas agrupan elementos solo cuando el cambio es literalmente el mismo en todos ellos.

ID	Elemento	Estado	Qué hacía antes	Qué hace ahora	Quién dependía y cómo se reconecta
E-01	Validaciones/ValidarRes.cs	Sale	Un if de cuatro condiciones con un texto único	Ya no existe	GuardadoValidado sigue pidiendo IValidador<Res>; la raíz le entrega el compuesto (E-11, E-12)
E-02	Contratos/IFabricaVacuna.cs	Entra	No existía	Creador: Tipo, TipoSoportado, Crear(SolicitudVacuna) y los tres textos observables	Lo consumen FabricaVacunas (E-14) y MapeadorVacuna (E-17)
E-03	Fabricas/FabricaVacunaBacteriana.cs y FabricaVacunaViva.cs (2)	Entra	El cuerpo de CrearBacteriana y CrearViva	Creadores concretos, dueños de sus textos	Se registran en la raíz (E-32): un tipo nuevo es 1 clase y 1 línea
E-04	Valores/SolicitudVacuna.cs	Entra	Seis parámetros sueltos por sobrecarga	Objeto de solicitud, con ConLote(. . .)	Unifica la firma de Crear; sin él haría falta una sobrecarga por tipo
E-05	Contratos/IEfectoVenta.cs	Entra	No existía	Estrategia: TipoSoportado y Aplicar(. . .)	Lo consume ServicioVenta.AplicarEfecto(:100-104)

ID	Elemento	Estado	Qué hacía antes	Qué hace ahora	Quién dependía y cómo se reconecta
E-06	Estrategias/EfectoVentaRetiroInventario.cs y EfectoVentaSinEfecto.cs (2)	Entra	La línea <code>...L_re-ses.Remove(res)</code> , sin alternativa	Retirar la res o no tocar el inventario	Se registran en la raíz: un artículo nuevo es 1 clase y 1 línea
E-07	Contratos/IArticuloVendible.cs	Entra	No existía	Declara lo único que la venta necesita: Nombre	La implementan Res (E-25) y ProductoDerivado (E-08)
E-08	Clases/ProductoDerivado.cs	Entra	No existía	Artículo vendible que no es animal: tipo, nombre y cantidad	Lo construye VentaController (E-29); es SC-1
E-09	Contratos/IPublicadorEvento.cs	Entra	No existía	Observador: Informar (ContextoAviso, IReceptorEventos)	Deja que Potrero, GestorReses y ServicioVacunacion reciban una colección
E-10	Valores/ContextoAviso.cs	Entra	Cada publicador recibía parámetros propios	Contexto compartido: potrero, cantidad de reses, res y los dos contadores	Precio declarado de unificar la firma
E-11	Validaciones/ValidadorCompuesto.cs	Entra	El if de cuatro condiciones	Recorre las reglas y corta en la primera que falla, con el literal congelado (:38-58)	Se compone en RaizComposicion.cs:91-97; el llamador no lo distingue de una hoja
E-12	Validaciones/ReglasRes/ — ReglaResNoNula, ReglaNombreObligatorio, ReglaPesoPositivo, ReglaEdadPositiva (4)	Entra	Los cuatro operandos del <code>\ \ </code>	Hojas: una condición por clase	El orden reproduce el del <code>\ \ </code> : ReglaResNoNula primera, porque las otras asumen res no nula
E-13	Views/Venta/VenderProducto.cshtml	Entra	No existía	Pantalla de venta de producto derivado	La sirve VentaController; única vista nueva
E-14	Servicios/FabricaVacunas.cs	Se transforma	Cuatro métodos fijos con validación repetida	Recibe IEnumerable<IFabricaVacuna> y resuelve en ObtenerFabrica (:116-117); de 10 973 a 8 321 bytes	VacunaService ahora llama a <code>Crear(tipoVacuna, ...)</code>
E-15	Servicios/VacunaService.cs	Se transforma	Un if/else sobre parámetros opcionales elegía el método	Recibe tipoVacuna y delega (:48, 61)	VacunaController le pasa el tipo, que antes se descartaba
E-16	Controllers/VacunaController.cs	Se transforma	if (tipoVacuna == "Bacteriana") ... else ...	Valida el formulario y pasa el tipo sin ramificar	El servicio absorbió la decisión; la vista no cambió
E-17	Infraestructura/MapeadorVacuna.cs	Se transforma	Cuatro ramas al leer el .txt	Diccionario FabricasPorTipo (:50) y <code>fabrica.Crear(...)</code> (:114,:178)	Registro propio: la persistencia se instancia fuera del contenedor
E-18	Servicios/ServicioVacunacion.cs	Se transforma	Creaba sus publicadores con new	Recibe PublisherVacunaVencida y la colección (:43) e itera (:131)	La raíz arma su lista a mano; no es la de GestorReses
E-19	Eventos/ — PublisherPesoMin, PublisherPesoVenta, PublisherPotreroLleno, PublisherPotreroMitad, PublisherVacunacionCompletada (5)	Se transforma	Cada uno con su método y su firma	Implementan IPublicadorEvento. Informar y leen del contexto solo lo suyo	Quien los disparaba con new ahora itera. PublisherVacunaVencida no se modificó (B-07)
E-20	Clases/Potrero.cs	Se transforma	Cuatro campos = new Publisher...() en secuencia escrita a mano	Recibe los avisos por parámetro en <code>anadir_res (:66)</code> e itera (:108)	Es entidad: se los pasa quien la llama, en el orden mitad → lleno → peso mínimo → peso de venta
E-21	Servicios/GestorReses.cs	Se transforma	Instanciaba publicadores en tres puntos	Recibe dos listas por constructor (:38) y las itera	Momentos distintos piden listas distintas; la raíz las arma

ID	Elemento	Estado	Qué hacía antes	Qué hace ahora	Quién dependía y cómo se reconecta
E-22	Infraestructura/RepositorioPotrerosArchivo.cs	Se transforma	Pasaba la política y las fábricas	Pasa además avisosAltaDeRes	Recibe los avisos reales, no una lista vacía
E-23	Servicios/ServicioVenta.cs	Se transforma	vender_res construía la venta y borraba la res en la misma línea	Recibe IEnumerable<IEfectoVenta> (:31); vender_res y vender_producto (:76) delegan en AplicarEfecto (:100)	Vender una res no cambió; vender_producto usa el mismo registro, no una rama nueva
E-24	Clases/Venta.cs	Se transforma	Campo Res obligatorio	Campo IArticuloVendible	Quien leía venta.Res lee venta.Articulo (B-10)
E-25	Clases/Res.cs	Se transforma	public abstract class Res	: IArticuloVendible	No gana un miembro: ya tenía Nombre; la línea de disco es idéntica
E-26	Validaciones/ValidarVenta.cs	Se transforma	if (... \\ \\ venta.Res == null \\ \\ ...)	if (... \\ \\ venta.Articulo == null \\ \\ ...)	Sigue monolítico: el Composite no se le aplicó. Deuda declarada
E-27	Infraestructura/MapeadorVenta.cs	Se transforma	7 columnas leídas de la res	DescomponerArticulo (:49-58) da la misma tupla de 7 para res o producto	Ninguna columna nueva: el producto reutiliza peso y tipo
E-28	Servicios/VentaService.cs	Se transforma	Solo sabía vender reses	Gana la operación de SC-1	ResService no se tocó
E-29	Controllers/VentaController.cs	Se transforma	Una única acción de venta	Gana la venta de derivado y sirve la vista nueva	Construye el ProductoDerivado desde el formulario
E-30	Views/Venta/Index.cshtml	Se transforma	Listaba ventas de reses	Distingue reses de productos	Lee la venta a través del artículo
E-31	Servicios/GuardadoValidado.cs	Se transforma	Cinco métodos con el mismo bucle	El bucle se unificó en Validar<T> (:88)	Conserva 7 dependencias; GuardarVacunasAplicadas mantiene su bucle anidado porque el orden es salida observable
E-32	Composicion/RaizComposicion.cs	Se transforma	Registraba repositorios, validadores y servicios	Registra además las dos fábricas, las dos estrategias, los cinco publicadores en orden y compone ValidadorCompuesto<Res> (:91-97); de 9 642 a 15 398 bytes	Único archivo donde se elige implementación en los cuatro patrones: un tipo nuevo cuesta 1 clase y 1 línea aquí

Estado	Clases e interfaces	Vistas	Total
Sale	1	0	1
Entra	16	1	17
Se transforma	22	1	23

7. Actividad 3.3 · Fichas de los patrones adoptados

Ficha 1 · Factory Method — catálogo de vacunas

Campo	Contenido
Patrón y dolor	Factory Method sobre P-01 : cuatro métodos fijos en FabricaVacunas.cs, if/else en VacunaService.cs:49-58 y VacunaController.cs:95-107, cuatro ramas en MapeadorVacuna.cs. Costo medido: 14 archivos por tipo nuevo
Alternativas	(a) No hacer nada : descartada, SC-3 abre una tercera familia sanitaria y el costo se repetiría entero. (b) Abstract Factory (B-03): una sola familia deja la interfaz con una sola implementación. (c) Builder (B-04): las sobrecargas cruzan dos ejes independientes y su API fluida cambiaría un mensaje de error observable
Qué sale y qué entra	Sale : los cuatro métodos fijos y las tres ramificaciones por tipo. Entra : IFabricaVacuna (creador, E-02), FabricaVacunaBacteriana y FabricaVacunaViva (creadores concretos, E-03) y SolicitudVacuna (E-04)
Cómo se relaciona	RaizComposicion construye las dos fábricas y se las inyecta a FabricaVacunas, que elige por discriminador en ObtenerFabrica (:116-117) con fábrica por defecto. MapeadorVacuna mantiene su propio registro (:50) porque la persistencia se construye fuera del contenedor. No interactúa con los otros tres patrones
Impacto	Creadas 4 , modificadas 5 (FabricaVacunas, VacunaService, VacunaController, MapeadorVacuna, RaizComposicion), eliminadas 0 . Anexo B : habilita SC-3; no toca SC-1 ni SC-2

Campo	Contenido
Qué cues- ta	Cubrió la creación, no el conteo: <code>ServicioVacunacion.cs:73-96, 118-122</code> sigue ramificando con <code>is</code> y <code>Res</code> sigue declarando un tope por tipo. Dos registros del mismo mapa que hay que mantener sincronizados. La fábrica por defecto convierte un discriminador desconocido en un objeto silencioso en vez de un error ruidoso
Origen	Idea nuestra, contra dos propuestas de la herramienta que rechazamos: <code>Abstract Factory (B-03)</code> y <code>Builder (B-04)</code>

Ficha 2 · Strategy — efecto de la venta sobre el inventario (habilita SC-1)

Campo	Contenido
Patrón y dolor	Strategy sobre P-02 , el punto de mayor costo: <code>ServicioVenta.cs:42-46</code> construía la venta y borraba la res en la misma línea, <code>Venta.cs:13, 16</code> exigía una <code>Res</code> , <code>ValidarVenta.cs:14</code> la rechazaba si era nula y <code>MapeadorVenta.cs:26</code> escribía sus datos. Costo de SC-1: 14 archivos
Alternati- vas	(a) No hacer nada: SC-1 era inviable sin tocar los 14 archivos. (b) Jerarquía de Venta por tipo (B-10): corregida, multiplicaría la persistencia y abriría columnas en <code>Ventas.txt</code> . (c) Un if sobre el tipo dentro de vender_res: es el condicional que P-02 identifica como el problema
Qué sale y qué entra	Sale: la línea <code>...L_reses.Remove(res)</code> dentro de <code>vender_res</code> y la exigencia de que una venta tenga una res. Entra: <code>IEfectoVenta (E-05)</code> , sus dos estrategias concretas (<code>E-06</code>), <code>IArticuloVendible (E-07)</code> y <code>ProductoDerivado (E-08)</code>
Cómo se relaciona	La raíz registra las dos estrategias; <code>ServicioVenta</code> las recibe por constructor (<code>:31</code>) y resuelve en <code>AplicarEfecto (:100-104)</code> por <code>TipoSoportado.IsAssignableFrom</code> , el mismo criterio que <code>IFabricaRes</code> usa en <code>Potrero.anadir_res.vender_res</code> y <code>vender_producto</code> usan el mismo registro
Impacto	Creadas 5 más la vista <code>VenderProducto.cshtml</code> ; modificadas 9 (<code>ServicioVenta</code> , <code>Venta</code> , <code>Res</code> , <code>ValidarVenta</code> , <code>MapeadorVenta</code> , <code>VentaService</code> , <code>VentaController</code> , <code>Views/Venta/Index.cshtml</code> , <code>RaizComposicion</code>); eliminadas 0 ; <code>ResService</code> no se tocó. Anexo B: implementa SC-1 completa
Qué cues- ta	La resolución es por tipo en ejecución: un artículo sin estrategia registrada hace que <code>First</code> lance donde antes avisaba el compilador (riesgo R-03). <code>MapeadorVenta</code> gana una conversión de ida y vuelta (<code>:49-58</code>) para meter dos formas de venta en las mismas 7 columnas
Origen	Propuesta aceptada tras verificarla (B-13) : confirmamos que <code>ServicioVenta.cs:46</code> era el único punto que retiraba la res. La elección de SC-1 es nuestra (B-01) y el modelado de Venta, una corrección nuestra (B-10)

Ficha 3 · Observer — publicadores de eventos del dominio

Campo	Contenido
Patrón y dolor	Observer sobre P-03 : 8 instanciaciones con <code>new</code> en 3 clases (<code>Potrero.cs:21-24, 105-108</code> , <code>GestorReses.cs:30-31, 77-78, 188-189</code> , <code>ServicioVacunacion.cs:29-30</code>), sin interfaz común y con la secuencia escrita a mano. Ninguna prueba podía sustituirlos
Alternati- vas	(a) No hacer nada: cambiar la política costaba 3 clases de dominio y 5 métodos, y las 8 instanciaciones eran intestables. (b) event/delegate de C#: el orden de un multicast no está garantizado por contrato y aquí el orden es salida observable. (c) Una interfaz para los seis (B-07): corregida a cinco
Qué sale y qué entra	Sale: las 8 instanciaciones y las seis firmas distintas de <code>Informar_*</code> . Entra: <code>IPublicadorEvento (E-09)</code> y <code>ContextoAviso (E-10)</code>
Cómo se relaciona	La raíz registra los cinco publicadores y arma a mano las listas de <code>GestorReses</code> (alta y alimentación) y <code>ServicioVacunacion</code> (una), porque cada lista es distinta y su orden es el orden de disparo. <code>Potrero</code> es una entidad y recibe la suya por parámetro en <code>anadir_res (:66)</code>
Impacto	Creadas 2 , modificadas 10 (los 5 publicadores, <code>Potrero</code> , <code>GestorReses</code> , <code>ServicioVacunacion</code> , <code>RepositorioPotrerosArchivo</code> , <code>RaizComposicion</code>), eliminadas 0 . Anexo B: no implementa solicitud, pero hace verificable el resto: el caso 19 agrega en vivo un aviso que ningún publicador conoce
Qué cues- ta	<code>ContextoAviso</code> es un contexto compartido del que cada publicador lee dos o tres campos e ignora el resto. Un publicador en la lista equivocada lee campos vacíos y calla sin error (riesgo R-05 , caso <code>PublisherPotreroMitad</code>). Depurar un aviso exige saber en qué lista está y en qué posición
Origen	Propuesta corregida en el alcance (B-07: cinco, no seis) y aceptada en el mecanismo de orden (B-14), tras comprobar que el orden de <code>Potrero.cs:105-108</code> es salida observable

Ficha 4 · Composite — reglas de validación de la res

Campo	Contenido
Patrón y dolor	Composite sobre P-05 : <code>ValidarRes.cs:19</code> metía nulidad, nombre, peso y edad en un solo <code>if</code> con un texto único (<code>ResultadoValidacion.cs:23</code>), y <code>GuardadoValidado.cs:49-65, 69-149</code> repetía cinco veces el mismo bucle con 7 dependencias
Alternati- vas	(a) No hacer nada: una regla nueva obliga a editar el <code>if</code> , y SC-1 y SC-3 traen agregados nuevos. (b) Chain of Responsibility (B-05): declara «el primero que pueda resuelve» y aquí aplican todas. (c) Decorator (B-15): el orden de anidamiento pasa a importar y el contrato envuelto está congelado. (d) Command (B-08): un método genérico unifica los cinco métodos sin patrón
Qué sale y qué entra	Sale: <code>ValidadorRes</code> entera (E-01), única clase eliminada del reto. Entra: <code>ValidadorCompuesto<T> (E-11)</code> y las cuatro hojas de <code>ReglasRes/ (E-12)</code>

Campo	Contenido
Cómo se relaciona	La composición y su orden se fijan en <code>RaizComposicion.cs:91-97</code> y solo ahí. <code>GuardadoValidado</code> sigue recibiendo un <code>IValidador<Res></code> y no distingue un compuesto de una hoja: esa indistinguibilidad es el patrón. Se relaciona con el Strategy de forma indirecta: <code>ValidadorVenta valida venta.Articulo (E-26)</code>
Impacto	Creadas 5 , modificadas 2 (<code>GuardadoValidado</code> , <code>RaizComposicion</code>), eliminadas 1 . De paso, el bucle repetido se unificó en <code>Validar<T> (:88)</code> . Anexo B : habilita SC-1 y SC-3
Qué cuesta	Cuatro archivos donde había cuatro operandos de un <code>\ \ </code> . El orden de registro pasa a ser semántico y frágil — <code>ReglaRes-NoNulla</code> debe ir primera— y nada en el compilador lo protege. Depurar una validación fallida exige recorrer la lista de la raíz
Origen	Propuesta corregida (B-05, de cadena a compuesto) y dos propuestas rechazadas (B-15 Decorator, B-08 Command)

Alcance declarado. Dos patrones cerraron su punto de dolor a medias. El Factory Method cerró la creación de vacunas pero no el conteo, porque el orden en que se acumulan los contadores decide el mensaje que ve el operario. El Composite se aplicó solo a `Res: ValidadorPotrero, ValidadorVacuna, ValidadorVenta` y `ValidadorChip` siguen resolviendo todo en un método único, porque ninguna solicitud del Anexo B obliga hoy a extenderlos y componerlos habría añadido diez clases sin punto de dolor que las justifique.

8. Actividad 4.1 · Matriz de verificación SOLID

Refuerza: el patrón resuelve exactamente el problema de ese principio. **Neutro:** no lo toca. **Tensionado pero compensado:** introduce un riesgo real, declarado junto con lo que lo compensa. **Roto:** no aplica en ninguna celda; no dejamos ningún principio roto sin compensación.

Patrón adoptado	SRP	OCP	LSP	ISP	DIP
Factory Method (P-01, catálogo de vacunas)	Refuerza	Refuerza	Tensionado pero compensado	Neutro	Refuerza
Strategy (P-02, efecto de venta / SC-1)	Refuerza	Refuerza	Tensionado pero compensado	Neutro	Refuerza
Observer (P-03, publicadores de eventos)	Refuerza	Refuerza	Tensionado pero compensado	Neutro	Refuerza
Composite (P-05, validadores de res)	Refuerza	Refuerza	Tensionado pero compensado	Neutro	Refuerza

SRP. Cada patrón sacó de un método una decisión que no le correspondía: qué tipo de vacuna construir salió de `FabricaVacunas`, qué le pasa al inventario salió de `vender_res`, qué avisos se disparan salió de `Potrero` y `GestorReses`, y qué condiciones definen una res válida salió de un `if`.

OCP. En los cuatro casos un tipo nuevo cuesta una clase y una línea en `RaizComposicion`, sin condicionales nuevos. Ninguna de las dos resoluciones —`ObtenerFabrica` y `AplicarEfecto`— crece con un `if` por tipo: las dos recorren un registro.

DIP. `FabricaVacunas`, `ServicioVenta`, `GestorReses` y `GuardadoValidado` dependen de `IFabricaVacuna`, `IEfectoVenta`, `IPublicadorEvento` e `IValidador<T>`; ninguno nombra una clase concreta. Las 8 instanciaciones con `new` dentro del dominio desaparecieron.

El hilo conductor de la columna LSP

Los cuatro patrones comparten la **misma** tensión: cada uno introduce una familia de implementaciones donde una implementación individual asume una entrada más estrecha de la que su contrato declara, que es precondition fortalecida. Ninguna está rota en el sistema entregado, porque la seguridad no la da la implementación sino **el único punto que la invoca**, que siempre le entrega una entrada compatible. Es una propiedad de la composición, y por eso queda declarada aquí y no escondida.

- **Factory Method.** `FabricaVacunaBacteriana.Crear` lanza si no llegó `PeriodoAplicacion` y `FabricaVacunaViva.Crear` exige `GradoAtenuacion`, pero `SolicitudVacuna` declara los dos campos opcionales. **Compensación:** `ObtenerFabrica` resuelve por el discriminador que ya viene explícito desde `VacunaController`, y este siempre completa el campo correspondiente antes de llamar. La guarda es defensiva y hoy inalcanzable — el mismo argumento que sostiene ADR-05 para `IFabricaRes`.
- **Strategy.** `EfectoVentaRetiroInventario.Aplicar` hace `(Res)articulo` y la firma no promete que el artículo sea una `Res`. **Compensación:** `AplicarEfecto` resuelve `_efectos.First(e => e.TipoSoportado.IsAssignableFrom(tipoArticulo))` antes de llamar, así que el cast siempre es seguro. Probado en `venderUnaResLaRetiraDelPotreroYVenderUnProductoNoTocaElInventario`.
- **Observer.** `PublisherPotreroMitad.Informar` lee `contexto.Potrero`; en `avisosAlimentacion`, donde ese campo nunca se llena, la condición numérica no coincidiría — funciona por el umbral de negocio, no por garantía del tipo. **Compensación:** la raíz registra cada publicador solo en la lista cuyo contexto llena los campos que lee, y el caso 19 demuestra que un aviso nuevo bien registrado se dispara sin tocar `Potrero.cs` ni `GestorReses.cs`.
- **Composite.** `ReglaNombreObligatorio`, `ReglaPesoPositivo` y `ReglaEdadPositiva` asumen `res != null`, aunque `IValidador<Res>.Validar` no exige no-nulidad. **Compensación:** `ValidadorCompuesto<Res>` corta en la

primera regla inválida y `ReglaResNoNula` se registra siempre primero. Probado en `ValidadorCompuestoCortaEnLaPrimeraReglaInvalida...` y `ElOrdenDeLasCuatroReglasDeResEvitaNullReferenceException`.

Por qué ISP queda Neutro y no Refuerza. Las cuatro interfaces ya nacían con un solo método relevante por implementación desde el Reto 1 (ADR-03). Ninguna implementación se ve forzada a escribir un método que no usa, pero tampoco había un contrato gordo que este trimestre haya partido; marcarlo «Refuerza» sería apuntarnos un mérito ajeno.

Error típico del Anexo A	¿Aplica aquí?
Fábrica que crece con un condicional por tipo nuevo	No — <code>ObtenerFabrica</code> y <code>AplicarEfecto</code> resuelven por registro, cero <code>if</code> por tipo
Fachada que absorbe lógica de negocio	No — <code>Facade</code> se evaluó y se descartó (B-06)
Punto de acceso global de instancia única	No — <code>Singleton</code> no se adoptó
Método plantilla con paso vacío en una subclase	No — <code>Template Method</code> no se adoptó
Envoltura que cambia el contrato de lo envuelto	No — <code>Decorator</code> se descartó precisamente por ese riesgo (B-15)

9. Actividad 4.2 · Evidencia de comportamiento preservado

El arnés de caracterización ejecuta el sistema completo sobre los datos históricos del cliente y vuelca mensajes, tipo de alerta, navegación y líneas de disco. Se corrió antes y después de cada patrón.

- **Los 15 casos heredados del Reto 1** corren sin modificar y producen **diff vacío** contra la salida guardada: `04-verificacion/salida-rediseñada.txt`.
- **Los 3 casos de SC-2** (chips, entregados el trimestre pasado) también dan **diff vacío**: `04-verificacion/salida-sc2.txt`.
- **4 casos nuevos** (19 a 22) en `04-verificacion/salida-patrones.txt`, uno o dos por patrón. No comparan contra el Reto 1 porque ejercitan comportamiento que el Reto 1 no tenía.
- **32 pruebas xUnit** en `src/Modificado/Pruebas (dotnet test)`; eran 26 al cierre del Reto 1. Las 6 nuevas prueban exactamente las compensaciones de LSP de §8.

Salidas antes y después, lado a lado

Caso	Salida AS-IS (Reto 1)	Salida TO-BE (con patrones)	¿Coincide?
Añadir res Pinta (edad 6, peso 100)	[mensaje] La res Pinta ha sido añadida al potrero Potrero_Terberos con éxito. · [Evento] La res 'Pinta' tiene un peso 100, está en desnutrición.. Datos válidos. Guardado exitoso en BD	idéntica	Sí
Alimentar Pinta con 149	[mensaje] La res 'Pinta' ha sido alimentada, ahora pesa 250 kg. · [Evento] ... apta para venta.	idéntica	Sí
Aplicar lote LB-A a Pinta	[mensaje] Vacuna aplicada correctamente a la res Pinta. [Evento] ... Bacterianas: 1, Vivas: 0. ...	idéntica	Sí
Vender Trueno por 2 500 000	[mensaje] Venta de la res Trueno realizada con éxito · línea de disco Potrero_Novillos\ <H0Y>\ Trueno\ 550\ 60\ Novillo\ 2500000	idéntica, incluida la línea de disco byte a byte	Sí
Round-trip de guardado sobre los datos históricos	6 archivos de datos	idénticos (04-verificacion/datos-rediseñado/)	Sí

Los cuatro casos nuevos

#	Patrón	Qué demuestra	Resultado
19	Observer	<code>AvisoDeAuditoria</code> se registra en la raíz de la prueba y se dispara al alimentar	[Auditoria] operación registrada, sin una línea de código en <code>Potrero.cs</code> ni <code>GestorReses.cs</code>
20	Strategy (SC-1)	Vender 150 litros de lácteo desde <code>Potro_Cebones</code>	[reses-antes] 226 / [reses-despues] 226: inventario intacto
21	Strategy	Contraste: vender la res Rayo del mismo potrero	La res sí se retira. Mismo <code>ServicioVenta</code> , sin un <code>if</code> que distinga los dos casos
22	Composite	Una res con edad 0	Se agrega en memoria pero no se guarda en disco, y el texto que ve el operario es el mismo <code>Invalido()</code> congelado

10. Actividad 5 · Registro de riesgos y plan de cambio

Escala, fijada antes de puntuar. *Probabilidad* — 1 exige violar a propósito una regla escrita · 2 solo si alguien nuevo toca sin leer la vista técnica · 3 ocurre en el próximo cambio ordinario · 4 ya ocurrió una vez en este repositorio · 5 está ocurriendo hoy. *Impacto* — 1 molestia local · 2 retrabajo de un archivo · 3 cambia una salida

observable, congelada por el encargo · 4 pérdida de datos recuperable · 5 pérdida silenciosa del histórico del cliente.

ID	Riesgo (si ocurre X, entonces Y)	P	I	Exp	Qué hacemos para evitarlo	Señal de que se está materializando
R-01	Si alguien añade un campo a una entidad que se guarda en disco, entonces una línea mal formada se descarta sin error y el histórico del cliente pierde registros sin que nadie lo note	3	5	15	P-06 queda fuera de alcance. Todo cambio en Infraestructura/ obliga a correr el arnés antes de integrarlo y a anexar la columna al final de la línea, nunca intercalada	Los archivos que deja el arnés en 04-verificacion/datos-rediseñado/ traen menos líneas que la corrida anterior
R-02	Si alguien corrige uno de los cinco defectos que el cliente ya conoce y que se conservan a propósito, entonces la pantalla del operario cambia sin autorización	3	3	9	Están fijados por pruebas escritas al revés: afirman que el sistema se comporta mal. Quien las vea fallar debe leer la decisión que las sostiene antes de tocar nada	Falla una de las cinco pruebas de Pruebas/PruebasDe-DeudaCongelada.cs
R-03	Si se agrega un artículo vendible nuevo y se olvida registrar qué le pasa al inventario al venderlo, entonces la venta no falla al compilar: falla delante del operario la primera vez que la intenta	3	3	9	El artículo y su efecto entran en el mismo cambio; la guía de dónde tocar lo declara como una sola fila, no dos	El operario recibe «Error inesperado en el método vender_producto» al vender el artículo nuevo
R-04	Si el diseño publicado nombra elementos que el código no tiene, entonces quien entre al equipo abre el archivo equivocado y el cambio cuesta el doble	4	2	8	La vista técnica se redacta leyendo el código, no los diagramas; antes de publicar se coteja nombre por nombre lo dibujado contra src/	Un nombre del diseño publicado no devuelve resultados al buscarlo en src/
R-05	Si un aviso se registra en la lista equivocada al ensamblar el sistema, entonces el operario deja de recibir esa alerta y nadie lo nota, porque el sistema no falla: calla	2	3	6	Cada aviso se registra solo en la lista cuyo contexto llena los datos que ese aviso lee; la restricción está escrita junto al registro	El caso 19 de salida-patrones.txt reporta menos avisos que antes
R-06	Si se reordenan los avisos en el punto de ensamblaje, entonces los mensajes le llegan al operario en otro orden, y ese orden es parte de lo que el cliente ya da por hecho	2	3	6	El orden —mitad, lleno, peso mínimo, peso de venta— es el de la colección registrada y no se toca sin autorización	git diff 04-verificacion/salida-rediseñada.txt deja de salir vacío tras correr el arnés

Evidencia. R-01: MapeadorRes.cs:103 descarta la línea con `return false` sin lanzar excepción ni dejar rastro, y son 8 archivos con la misma forma. R-02: dos de los cinco defectos —un alta correcta de usuario que se reporta como error, un lote duplicado que se reporta como éxito— se leen como errores evidentes, así que el riesgo de que alguien los «arregle» de buena fe no es teórico. R-03: ServicioVenta.cs:103 resuelve con `First`, que lanza en ejecución si ningún efecto corresponde al artículo. **R-04 se puntúa en 4 porque ya se materializó dos veces en esta entrega:** el Composite entró en el commit d809d6d y el diagrama se publicó 21 horas después (2bd8a36) dibujando ValidadorRes, la clase que ese mismo commit había eliminado. Las dos veces la detectó la señal de la tabla, y la corrección es previa a publicar este documento.

Plan de cambio

Un patrón por entrega, en orden de riesgo creciente. (1) *Catálogo de vacunas*: replica un mecanismo que el sistema ya tenía funcionando para el ganado. (2) *Efecto de venta*: trae la solicitud autorizada, y llega cuando el equipo ya practicó con el primero. (3) *Avisos del dominio*: toca muchos archivos pero no agrega comportamiento, así que su verificación es un diff que debe salir vacío. (4) *Validación por reglas*: último, es el único que elimina un archivo existente.

Puerta de verificación entre paso y paso: pruebas en verde, arnés corrido y diff de las salidas guardadas saliendo vacío. Si el diff no sale vacío, el paso no entra. **Reversión:** cada patrón es un cambio aislado en la historia del repositorio, así que deshacer uno no arrastra a los otros tres; esa es la razón de no haberlos integrado juntos. **Del negocio necesitamos** que la autorización de vender productos derivados siga en pie y que ninguna otra solicitud entre mientras dure la adopción.

11. Actividad 6 · Vista para el negocio

De qué sistema hablamos. Es el programa con el que la hacienda lleva el día a día del ganado: el personal entra desde el navegador y registra los potreros, los animales de cada uno con su nombre, peso y edad, las vacunas que se les aplican y las ventas. Todo queda guardado en archivos dentro del propio servidor de la finca. Lo que aporta, más allá de guardar datos, es que conoce las reglas de la hacienda y avisa solo: cuando un potrero va por la mitad, cuando se llena, cuando un animal está por debajo de su peso saludable, cuando alcanza el peso de venta, cuando termina su plan de vacunación y cuando una vacuna se vence.

Qué le vamos a hacer y qué no cambia. Cambiamos cómo está armado el programa por dentro. Por fuera no cambia nada: los mismos mensajes, las mismas pantallas, los mismos datos guardados y las mismas reglas.

Quien usa el sistema todos los días no va a notar la diferencia, y eso lo comprobamos ejecutando el programa antes y después y comparando lo que imprime, línea por línea. Lo único nuevo es lo que el negocio pidió y autorizó: la hacienda ya puede vender leche, carne y piel, no solo el animal completo.

Dónde se está yendo hoy el tiempo y el dinero. El costo no está en escribir lo que se pide, sino en encontrar todos los sitios donde hay que escribirlo. Hoy, dar de alta un tipo de vacuna nuevo obliga a revisar **catorce puntos distintos**, y ninguno avisa si alguien se olvida de otro: el programa arranca igual y la falla aparece semanas después, cuando alguien nota que un animal nunca apareció en un listado. Vender un producto que no fuera el animal completo costaba otros catorce, y por eso esa petición llevaba tiempo sin poder atenderse.

Qué gana el negocio.

	Antes	Después
Un tipo de vacuna nuevo	14 puntos que revisar	1
Vender un producto nuevo	14 puntos que revisar	1
Una regla nueva sobre los animales	Reescribir una regla existente	1 pieza aparte
Un aviso nuevo para el operario	3 sitios del programa	1

Dos ganancias, y la segunda importa más. La obvia es que el trabajo se acorta. La que cuenta es que **el sitio donde hay que tocar es siempre el mismo y está señalado**: hoy una petición así no se puede prometer para una fecha, porque nadie sabe si va a aparecer un punto olvidado; a partir de ahora sí.

Qué cuesta. El programa tiene ahora más piezas, cada una más pequeña: es más fácil cambiar una, y hay más que conocer al llegar. Seguir la pista de un error cuesta un paso más, porque hay que mirar dónde ocurre y también dónde se decidió que ocurriera, que ahora son sitios distintos. Y todo el armado pasa por un único punto del programa: es la razón de que los cambios sean baratos, y también un sitio que hay que vigilar, porque si crece sin control se convierte en el próximo cuello de botella.

Qué riesgos hay. El más caro: **la forma en que el programa guarda la información en disco no se tocó**, porque cambiarla obligaba a reescribir los datos históricos de la hacienda; mientras siga así, si alguien añade un dato nuevo a una ficha sin el cuidado debido se pueden perder registros viejos sin que el programa se queje, y nos enteramos porque los archivos de datos quedan con menos líneas que antes. Hay **cinco fallas menores que el cliente ya conoce** y que se conservan tal cual, porque corregirlas cambiaría lo que el operario ve y eso no está autorizado; dejamos avisos automáticos que suenan si alguien las corrige por su cuenta. Y un riesgo silencioso: **el operario puede dejar de recibir una de las alertas sin que el programa falle**, porque un aviso mal conectado no da error, simplemente calla; por eso el orden y la conexión de los avisos quedaron por escrito.

Qué necesitamos y qué pasa si no se hace. Poco y nada costoso: que la autorización para vender productos derivados siga en pie y que no entre otra petición mientras dura el cambio; corregir alguna de las cinco fallas conocidas necesitaría una autorización expresa. Si no se hace, la petición de vender productos derivados sigue sin poder atenderse —no era cuestión de tiempo, era que el programa daba por hecho que lo único vendible era el animal entero—, la siguiente costará lo mismo que costaba ésta, y el riesgo de perder registros históricos crece con cada dato nuevo que se agregue.

Qué entendió una persona ajena al equipo. Presentamos esta vista a una persona sin formación técnica y le pedimos que la resumiera con sus palabras; la grabación va en el video. Explicó que al programa le van a cambiar la organización por dentro sin que cambie lo que se ve en pantalla, que lo nuevo es poder vender leche, carne y piel, y que la ganancia es que los cambios que antes obligaban a revisar catorce sitios ahora se hacen en uno. Del riesgo retuvo el de los datos guardados. No supo repetir el de los avisos, así que reescribimos ese párrafo para nombrar primero la consecuencia —el operario deja de recibir una alerta— y después la causa.

12. Actividad 6 · Vista para el equipo de desarrollo

Documento de consulta, no de lectura corrida: se busca la fila que se parece al cambio y se siguen las tres columnas.

La guía de dónde tocar

Tu cambio	Crear	Modificar	No tocar
Un tipo de vacuna nuevo	1 clase que implemente IFabricaVacuna	1 línea en RaizComposicion	FabricaVacunas, VacunaService, VacunaController, MapeadorVacuna
Un tipo de res nuevo	1 subclase de Res + 1 clase IFabricaRes	1 línea en RaizComposicion; el diccionario de ResService y 2 vistas, que son presentación	El resto del dominio

Tu cambio	Crear	Modificar	No tocar
Vender un artículo nuevo (SC-1: leche, carne y piel ya están)	1 clase IArticuloVendible; si su efecto sobre el inventario difiere de los dos que hay, 1 clase IEfectoVenta	1 línea en RaizComposicion por cada una	ServicioVenta
Un campo nuevo en algo que se guarda en disco	—	El mapeador y el repositorio de esa entidad, en Infraestructura/	El orden de las columnas que ya existen
Un agregado nuevo (SC-3: historia clínica)	1 clase de dominio, sus reglas IValidador<T>, 1 repositorio	GuardadoValidado y su constructor, que hace recompilar 3 servicios	Los 4 validadores que ya existen
Una regla de validación nueva	1 clase que implemente IValidador<T>	1 línea en el arreglo del ValidadorCompuesto correspondiente	El validador existente
Un aviso nuevo	1 clase que implemente IPublicadorEvento	1 línea en la lista de avisos que corresponda	Potrero, GestorReses, ServicioVacunacion

Las tres solicitudes del Anexo B caen en la tercera fila (vender derivados, implementada), la cuarta (chips, ya entregada: añadió 4 columnas al final de la línea) y la quinta (historia clínica).

Dónde se ensambla el sistema

Composicion/RaizComposicion.cs, 241 líneas. Es el **único** lugar donde se elige qué implementación satisface cada contrato, y por eso casi todas las filas de arriba dicen «1 línea»: es esa. La regla que lo sostiene: ninguna clase fuera de Composicion/ recibe IServiceProvider ni pide instancias en ejecución; todo entra por constructor. Si te ves pidiéndole una instancia al contenedor desde un servicio, estás reintroduciendo el problema que este diseño eliminó.

Los cuatro patrones: dónde vive cada uno

Patrón	Contrato	Implementaciones	Quién decide cuál se usa
Factory Method	IFabricaVacuna	FabricaVacunaBacteriana, FabricaVacunaViva	FabricaVacunas, por el tipo que llega del controlador
Strategy	IEfectoVenta	EfectoVentaRetiroInventario, EfectoVentaSinEfecto	ServicioVenta, buscando la que soporta el tipo del artículo
Observer	IPublicadorEvento + ContextoAviso	las cinco clases de Bib_Hacienda/Eventos/	El orden de la lista registrada en la raíz
Composite	IValidador<T>	ValidadorCompuesto<T> agrupando las reglas de ReglasRes/	La composición escrita en la raíz

No se relacionan entre sí: se relacionan con el mismo sitio. Los cuatro trasladaron a la raíz una decisión que antes era un condicional dentro de un método, y por eso un tipo nuevo en cualquiera de las cuatro familias cuesta una clase y una línea.

Reglas que no se rompen, y por qué

- La salida observable está congelada:** texto de los mensajes, tipo de alerta, navegación y formato de los archivos de datos. Se verifica con el arnés; el diff contra 04-verificacion/ debe salir vacío.
- El orden de los avisos es salida observable** —mitad, lleno, peso mínimo, peso de venta—: lo produce el orden de la lista en la raíz y ningún código lo protege.
- Cada aviso va solo en la lista cuyo contexto llena los campos que ese aviso lee.** PublisherPotreroMitad lee la cantidad de reses y el potrero: en la lista de alimentación callaría sin error.
- Las columnas nuevas se anexan al final de la línea, nunca intercaladas,** o se reescribe el histórico del cliente. MapeadorRes pasó de 5 a 9 columnas así.
- Los cinco defectos de PruebasDeDeudaCongelada se conservan.** Esas pruebas afirman que el sistema se comporta mal y pasan porque se comporta mal: si una falla, cambiaste algo que el operario ve.
- Nada de frameworks que resuelvan el problema:** el generador de proxies del sistema original se desmontó a propósito y no vuelve.

Deuda que queda declarada

- El catálogo de tipos de potrero (enum l_tipos_potreros) se guarda literalmente en disco: eliminarlo obligaría a reescribir el archivo de datos del cliente (P-04).
- La persistencia lee por posición de columna en 4 mapeadores y 4 repositorios, y una línea corta se descarta sin excepción. Es el riesgo de mayor exposición del registro (P-06, R-01).
- GuardadoValidado conserva sus 7 dependencias por constructor, y GuardarVacunasAplicadas mantiene su bucle anidado porque el orden de recorrido decide el último resultado registrado, que es salida observable.

- El catálogo de vacunas se resuelve por fábrica al **crear**, pero al **contar** sigue preguntando por el tipo con is (ServicioVacunacion.cs:73-96,118-122), y Res conserva un tope por tipo.
- El agrupamiento de reglas se aplicó solo a Res: ValidadorPotrero, ValidadorVacuna, ValidadorVenta y ValidadorChip siguen resolviendo todo en un condicional único, así que en ellos la fila «una regla nueva» todavía no es cierta.
- PublisherVacunaVencida queda fuera del patrón a propósito: devuelve un booleano del que depende si se lanza una excepción, así que es guarda de flujo y no aviso.